

Runbook Operacional

Radar de Tendências Tecnológicas — SENAI Futuro — IA

Runbook Operacional — Radar de Tendências Tecnológicas

Documento operacional: como implantar, operar, monitorar e recuperar o sistema em produção. Complementa (não substitui) docs/architecture.md (decisões de design) e docs/critical-review.md (avaliação crítica). Última atualização: 2026-07-07.

1. Visão geral do ambiente

Item	Valor
Aplicação	radar-tendencias-app (Azure App Service, Linux, plano B1)
Região	brazilsouth (App Service, Foundry e Cosmos DB colocalizados)
Resource Group	radar-trends
Runtime	Python 3.11, Streamlit (app/Home.py), servido via Tornado (não Gunicorn)
Persistência	Azure Cosmos DB serverless, database radar, container reports
IA	Azure AI Foundry (omc-cli/omc-ccg-cli), deployment gpt-5-radar (gpt-5.4-mini)
URL de produção	https://radar-tendencias-app.azurewebsites.net
Health check	GET /_stcore/health (endpoint nativo do Streamlit)

Diagrama de componentes completo em docs/architecture.md e docs/apresentacao/criterios/01-arquitetura.drawio.png.

2. Ambientes

Ambiente	Como ativar	Uso
Local (online)	.env preenchido + az login	Desenvolvimento normal, contra Foundry/Cosmos reais
Local (offline)	.env com RADAR_OFFLINE=1	Demo sem rede — troca

Ambiente	Como ativar	Uso
		CosmosReportRepository por LocalReportRepository (JSON em data/reports/)
Produção	App Service com App Settings (§4)	Único ambiente publicamente acessível

Não há ambiente de staging — o volume e o prazo do projeto não justificam um plano B1 adicional (Princípio II da constituição, simplicidade orientada a prazo).

3. Deploy

3.1 Pré-requisitos

- az login com acesso ao resource group radar-trends.
- Testes locais passando: pytest (50 testes, mockado, sem custo).
- Lint limpo: ruff check ..

3.2 Procedimento

```
cd research-hub
zip -r ../deploy.zip . -x ".venv/*" -x ".git/*" -x "data/reports/*.json" \
  -x "__pycache__/*" -x "*/__pycache__/*" -x ".pytest_cache/*" -x ".ruff_cache/*" \
  -x ".env" -x "tests/fixtures/_tmp_*/" -x ".understand-anything/*"

az webapp deploy -n radar-tendencias-app -g radar-trends --src-path ../deploy.zip --type zip
```

Provisionamento completo de infraestrutura (do zero) está em infra/provision.md — este runbook assume que Cosmos DB, App Service e Managed Identity já existem.

3.3 ⚠ Como interpretar o resultado do deploy — achado real recorrente

O CLI (az webapp deploy) relatou falso `504 Gateway Timeout` em múltiplas ocasiões reais (sessões de 2026-07-07) enquanto o deploy **terminava com sucesso do lado do servidor** minutos depois. Causa: o build do Oryx (instalação de dependências do requirements.txt) leva ~4-5 minutos quando uma

dependência nova é adicionada (ex.: `plotly`), excedendo o timeout de espera do próprio comando CLI — mas o processo continua rodando no Kudu e conclui normalmente.

Não confie apenas no código de saída do `az webapp deploy`. Sempre confirme o status real:

```
az webapp log deployment list -n radar-tendencias-app -g radar-trends \
  --query "sort_by([], &received_time)[-1]" -o json
```

Um deploy bem-sucedido mostra `"active": true, "complete": true, "status": 4`. Em seguida, confirme com o health check:

```
curl -s https://radar-tendencias-app.azurewebsites.net/_stcore/health
```

Resposta `200 OK` (corpo `ok`) confirma que o app está de fato servindo a versão nova.

3.4 Falhas conhecidas do deploy e como reagir

Sintoma	Causa provável	Ação
<code>ConnectionResetError(10054)</code> logo no início, antes de "Warming up Kudu"	Interferência local de antivírus/firewall com inspeção SSL na rota para <code>management.azure.com</code> (frequentemente uma rota IPv6 quebrada nesse trecho específico)	Testar <code>curl -4 -v https://management.azure.com</code> vs <code>curl -6 -v ...</code> ; se só o IPv6 falhar, rebaixar a precedência IPv6 localmente: <code>netsh interface ipv6 set prefixpolicy ::ffff:0:0/96 45 4</code> (reversível: repetir com 354). Não é um problema do Azure nem do código.
<code>504 Gateway Timeout</code> do Kudu, chegando a "Warmed up Kudu instance successfully" antes de falhar	Falso alarme do CLI — build do Oryx mais longo que o timeout do comando (ver §3.3)	Aguardar 1-2 min e confirmar via <code>az webapp log deployment list</code> (não retentar às cegas)
Deploy realmente falha (<code>status</code> $\neq$ 4, sem "Deployment successful" no log)	Erro real de build/dependência	<code>az webapp log deployment show -n radar-tendencias-app -g radar-trends</code> para o log completo do Oryx

3.5 Rollback

Não há pipeline de rollback automatizado. Para reverter:

```
git checkout <commit-anterior> -- .  
# refazer o zip e reimplantar conforme §3.2  
git checkout <branch-atual> -- .
```

Alternativa mais segura: `git revert` do commit problemático e reimplantar a partir da branch principal, evitando checkout parcial manual.

4. Configuração (App Settings / variáveis de ambiente)

Fonte de verdade do schema: `src/radar/config.py` (pydantic-settings).

Variável	Obrigatória	Descrição
PROJECT_ENDPOINT	Sim (produção)	Endpoint do projeto Azure AI Foundry
MODEL_DEPLOYMENT_NAME	Não (default <code>gpt-5-radar</code>)	Nome do deployment do modelo
COSMOS_ENDPOINT	Sim (produção)	Endpoint do Cosmos DB
COSMOS_KEY	Sim (produção)	Chave primária do Cosmos DB — ver §6.1, incidente real
MAX_ANALYSES_PER_DAY	Não (default 10)	Freio de custo — limite diário de análises persistidas
ANALYSIS_BUDGET_SECONDS	Não (default 360)	Orçamento de tempo total de uma análise
RADAR_OFFLINE	Não (default false)	1 troca Cosmos por repositório local em disco
OPENALEX_MAILTO	Não	E-mail para identificação educada na API OpenAlex

Verificação de integridade dos App Settings (não basta checar presença do nome):

```
az webapp config appsettings list -n radar-tendencias-app -g radar-trends \
```

```
--query "[].{name:name, valueLength:length(value)}" -o table  
# COSMOS_KEY deve ter ~88 caracteres – nunca 0 (ver incidente §6.1)
```

5. Monitoramento e verificação de saúde

5.1 Verificação manual pós-deploy (checklist)

1. Health check: `curl -s https://radar-tendencias-app.azurewebsites.net/_stcore/health` → 200.
2. Home carrega: `curl -s -o /dev/null -w "%{http_code}" https://radar-tendencias-app.azurewebsites.net/` → 200.
3. Histórico carrega sem erro de autorização do Cosmos: `.../Historico` → 200 (um 401/erro de autorização aqui é sintoma do incidente §6.1).
4. Rodar 1 análise real de smoke test (`infra/smoke_test.py` ou `infra/gen_test_report.py`) para validar o pipeline ponta a ponta contra Foundry/Cosmos reais.

5.2 Logs de runtime

```
az webapp log tail -n radar-tendencias-app -g radar-trends
```

Application Logging (File System) deve estar habilitado no portal (Configuração → Monitoramento → Log do aplicativo) para este comando retornar algo além do access log do servidor web.

5.3 Métricas de custo (gap conhecido)

`Report.metrics` não está populado hoje (achado B2 do code review, documentado em `docs/critical-review.md`) — não há breakdown de tokens/custo por etapa gravado no documento. Custo real deve ser confirmado via **Azure Cost Management** (escopo `radar-trends + ai-models-resource`), não estimado a partir do código.

6. Incidentes reais e runbooks de resposta

Esta seção documenta apenas incidentes **reais já ocorridos**, com causa raiz confirmada — não cenários hipotéticos.

6.1 COSMOS_KEY vazio → 401 Unauthorized no Histórico (2026-07-06)

Sintoma: CosmosHttpResponseError (Unauthorized): Required Header authorization is missing ao abrir a página Histórico em produção.

Causa raiz: durante o provisionamento, `az webapp config appsettings set ... COSMOS_KEY="$ (az cosmosdb keys list ...)"` foi executado numa sessão com `ConnectionResetError` recorrente; o comando interno retornou vazio, e o comando externo gravou `COSMOS_KEY=""` sem erro aparente — o setting aparecia como "presente" numa checagem que só olhava o nome.

Resposta:

```
NEW_KEY=$(az cosmosdb keys list -n cosmos-radar-tendencias -g radar-trends --query
primaryMasterKey -o tsv)
echo -n "$NEW_KEY" | wc -c # confirmar ~88 caracteres ANTES de gravar
az webapp config appsettings set -n radar-tendencias-app -g radar-trends --settings
COSMOS_KEY="$NEW_KEY"
az webapp config appsettings list -n radar-tendencias-app -g radar-trends \
--query "[?name=='COSMOS_KEY'].{valueLength:length(value)}" -o table # confirmar DEPOIS
também
```

Lição operacional: para qualquer segredo obtido via `$(...)` em um comando composto, sempre verificar o **comprimento do valor**, nunca apenas a presença do nome do setting.

6.2 Redeploy bloqueado por ConnectionResetError na rota SCM/Kudu (múltiplas sessões)

Sintoma: `az webapp deploy` (e comandos relacionados como `az webapp restart`, `az webapp log deployment show`) falham repetidamente com `ConnectionResetError(10054)` durante o handshake TLS, mesmo com outras chamadas Azure (`Cosmos`, `az account show`) funcionando normalmente na mesma sessão.

Diagnóstico que funcionou: testar `curl -v` diretamente (fora do `az CLI`) contra `management.azure.com`, isolando IPv4 de IPv6:

```
curl -4 -v --max-time 10 https://management.azure.com # se OK...
curl -6 -v --max-time 10 https://management.azure.com # ...e este falhar no handshake TLS
```

Isso confirma uma rota IPv6 local quebrada especificamente para esse destino — não um problema do Azure, nem necessariamente do antivírus (embora inspeção SSL de AV/firewall seja a causa mais comum desse padrão em geral).

Resposta: aplicar `netsh interface ipv6 set prefixpolicy ::ffff:0:0/96 45 4` (eleva a precedência de IPv4 mapeado acima do IPv6 nativo, sem desativar IPv6 — reversível com `35 4`). Se a

causa raiz for diferente (AV com inspeção SSL, por exemplo), a mitigação é tentar novamente de outra rede/máquina — o comando de deploy em si está correto, documentado em `infra/provision.md` §5.

Não fazer: re-executar o mesmo comando repetidamente sem diagnosticar — depois de 2-3 tentativas idênticas falhando com o mesmo erro, parar e investigar (ou escalar ao usuário) em vez de continuar tentando às cegas.

6.3 Rate limit real do modelo (429) sob carga normal (2026-07-05, smoke test T036)

Sintoma: `openai.RateLimitError` durante a síntese de uma análise legítima (não sob ataque nem uso concorrente anormal).

Causa raiz: capacidade do deployment `gpt-5-radar` estava deliberadamente baixa (10 unidades, por design de isolamento de custo — R10 em `research.md`) — 4 buscas web concorrentes + 1 síntese sobre o corpus consolidado excedem 10 unidades mesmo em uso normal.

Resposta aplicada: aumentar a capacidade do deployment (10 → 50 → 200 unidades), mantendo um teto explícito e moderado (não "ilimitado") — ainda muito abaixo da cota da assinatura na região. Esta é uma mudança de configuração de infraestrutura, **não exige redeploy de código**.

```
az cognitiveservices account deployment update -n omc-cli -g ai-models-resource \
  --deployment-name gpt-5-radar --sku-capacity 200
```

Correção de código associada: o orquestrador só tratava `TimeoutError`/`SynthesisError` na etapa de síntese — um `openai.RateLimitError` cru escapava e derrubava o processo. Corrigido para tratar qualquer exceção da síntese como `status=partial` (ver `src/radar/orchestrator.py`, bloco `except Exception`).

6.4 arXiv migrou para HTTPS, coleta acadêmica degradando 100% das vezes (2026-07-05)

Sintoma: `ArxivCollector` sempre retornava degradado no smoke test contra produção.

Causa raiz: `http://export.arxiv.org` passou a devolver 301 `Redirect` para `https://`; o cliente `httpx.AsyncClient` do projeto não segue `redirect` por padrão.

Resposta: URL base do coletor trocada diretamente para `https://` (`src/radar/collectors/arxiv.py`). Sem impacto de infraestrutura — correção de código pura, redeployada.

7. Freios de custo e abuso (R10)

Aplicação pública sem autenticação — estes três freios são deliberados, não um esquecimento:

5. **Access Restrictions por IP** — liberar apenas o IP do apresentador no dia da demo (o IP muda conforme o local, então isso é feito na hora, não com antecedência): ```bash az webapp config access-restriction add -n radar-tendencias-app -g radar-trends \ --rule-name allow-apresentador --action Allow --ip-address <SEU_IP>/32 --priority 100 ```

6. **Limite diário de análises** (`MAX_ANALYSES_PER_DAY`, default 10) — verificado no orquestrador antes de iniciar qualquer pipeline pago.

7. **Budget alert no Azure Cost Management** (~US\$30/mês) — criado manualmente no portal (`az consumption budget create tem bug reproduzível de API nesta assinatura`), com alerta em 80%/100%.

Limitação residual conhecida: as chamadas de classificação de escopo (`scope_guard.py`) não são, elas mesmas, limitadas pelo `MAX_ANALYSES_PER_DAY` (que só conta relatórios persistidos) — registrado como evolução futura menor.

8. Testes

```
pytest          # unit + integração, tudo mockado – sem custo, sem credenciais (50 testes)
pytest -m live   # smoke test contra Azure real – exige .env válido, consome tokens/RU
ruff check .     # lint
```

Rodar `pytest -m live` (ou `infra/smoke_test.py`) sempre que houver mudança em prompts de agente, parsing de resposta do LLM, ou integração com coletores — esses são os pontos onde bugs reais só apareceram contra APIs reais, nunca em mocks (ver §6.3, §6.4).

9. Referências

- Provisionamento completo (infraestrutura do zero): `infra/provision.md`
- Decisões de arquitetura e justificativas: `docs/architecture.md`
- Avaliação crítica contínua (vieses, limitações, custos, evoluções): `docs/critical-review.md`
- Decisões técnicas em formato ADR (Decision/Rationale/Alternatives): `specs/001-radar-tendencias/research.md`

- Grafo de conhecimento do código-fonte (navegável): `.understand-anything/knowledge-graph.json`